

13.08 – Aula do Bruno PDM

main() e MeuApp — todo app Flutter começa em *main()*, que chama *runApp()*. O **MeuApp** é a "casca" do app: define tema, título, e qual tela abrir primeiro (home). Ele é *StatelessWidget* porque nunca muda sozinho.

```
void main() {  
  runApp(const MeuApp());  
}
```

Classe Aluno — é só um "molde" de dados. Em vez de ficar com listas separadas (*nomes = [], idades = []...*), a gente guarda cada aluno como um pacotinho com nome, idade e curso juntos. Isso deixa o código mais organizado e evita bugs de "esqueci de atualizar uma das listas".

```
// "Modelo" do aluno: uma classe simples que agrupa nome, idade e curso  
// numa única "caixinha" de dados, em vez de usar 3 listas soltas.  
class Aluno {  
  final String nome;  
  final String idade;  
  final String curso;  
  
  Aluno({required this.nome, required this.idade, required this.curso});  
}
```

CadastroAlunosPage (StatefulWidget) — é a tela em si. Precisa ser "Stateful" porque toda vez que um aluno é cadastrado, a **tela precisa se redesenhar** pra mostrar o novo item na lista. Um *StatelessWidget* não consegue fazer isso sozinho.

```
class CadastroAlunosPage extends StatefulWidget {  
  const CadastroAlunosPage({super.key});  
  
  @override  
  State<CadastroAlunosPage> createState() => _CadastroAlunosPageState();  
}
```

TextEditingController — pense neles como "microfones" plugados em cada campo de texto. Enquanto o usuário digita, o *controller* vai guardando esse valor. Quando você quer saber o que a pessoa escreveu, você lê *controller.text*.

```
class _CadastroAlunosPageState extends State<CadastroAlunosPage> {  
  // Controllers: "sensores" que leem o que foi digitado em cada campo.  
  final TextEditingController _nomeController = TextEditingController();  
  final TextEditingController _idadeController = TextEditingController();  
  final TextEditingController _cursoController = TextEditingController();  
}
```

_cadastrarAluno() — é a função executada quando clica no botão. Ela:

```
// Função chamada quando o botão "Cadastrar" é clicado.
void _cadastrarAluno() {
  final nome = _nomeController.text.trim();
  final idade = _idadeController.text.trim();
  final curso = _cursoController.text.trim();
```

1. Lê o texto de cada *controller*.
2. Confere se nada está vazio (validação simples).
3. Usa `setState(() {...})` pra adicionar o novo Aluno na lista `_alunos` e avisar o *Flutter* que a tela precisa ser redesenhada. Esse `setState` é essencial — sem ele, o dado até entra na lista, mas a tela não atualiza visualmente.

```
// setState avisa o Flutter: "algo mudou, redesenhe a tela".
setState(() {
  _alunos.add(Aluno(nome: nome, idade: idade, curso: curso));

  // Limpa os campos depois de cadastrar, pra facilitar o próximo.
  _nomeController.clear();
  _idadeController.clear();
  _cursoController.clear();
});
```

4. Limpa os campos (`.clear()`) pra facilitar cadastrar o próximo.

dispose() — quando a tela é fechada/destruída, é boa prática "desligar" os *controllers* para não desperdiçar memória. Não é obrigatório para a atividade funcionar, mas é considerado boa prática em qualquer código *Flutter*.

```
// Boa prática: limpar os controllers quando a tela for destruída,
// evitando consumo de memória desnecessário.
@override
void dispose() {
  _nomeController.dispose();
  _idadeController.dispose();
  _cursoController.dispose();
  super.dispose();
}
```

O `build()` (a parte visual):

- *Scaffold* é a estrutura básica de tela (com barra superior, corpo, etc).

```
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(  
      title: const Text('Cadastro de Alunos'),  
    ), // AppBar
```

- *Column* empilha os widgets de cima pra baixo: os 3 *TextFields*, o botão e a

```
child: Column(  
  children: [  
    // Campo Nome  
    TextField(  
      controller: _nomeController,  
      decoration: const InputDecoration(  
        labelText: 'Nome',  
        border: OutlineInputBorder(),  
      ), // InputDecoration  
    ), // TextField
```

lista.

- Cada *TextField* está ligado a um *controller*, que é como ele "sabe" o que foi digitado.

```
// Campo Nome  
TextField(  
  controller: _nomeController,  
  decoration: const InputDecoration(  
    labelText: 'Nome',  
    border: OutlineInputBorder(),  
  ), // InputDecoration  
) // TextField  
const SizedBox(height: 12),  
  
// Campo Idade  
TextField(  
  controller: _idadeController,  
  keyboardType: TextInputType.number, // abre teclado numérico  
  decoration: const InputDecoration(  
    labelText: 'Idade',  
    border: OutlineInputBorder(),  
  ), // InputDecoration  
) // TextField  
const SizedBox(height: 12),  
  
// Campo Curso  
TextField(  
  controller: _cursoController,  
  decoration: const InputDecoration(  
    labelText: 'Curso',  
    border: OutlineInputBorder(),  
  ), // InputDecoration  
) // TextField  
const SizedBox(height: 16),
```

- *ElevatedButton* é o botão — o *onPressed: _cadastrarAluno* diz "quando clicado, execute aquela função".

```
// Botão Cadastrar
PreferredSize(
  width: double.infinity,
  child: ElevatedButton(
    onPressed: _cadastrarAluno,
    child: const Padding(
      padding: EdgeInsets.symmetric(vertical: 12),
      child: Text('Cadastrar'),
    ), // Padding
  ), // ElevatedButton
), // PreferredSize
const PreferredSize(height: 20),
```

- *Expanded* + *ListView.builder* mostram a lista de alunos. O *ListView.builder* é eficiente porque só constrói os itens que estão visíveis na tela (importante se a lista crescer muito).

```
Expanded(
  child: _alunos.isEmpty
    ? const Center(
        child: Text(
          'Nenhum aluno cadastrado ainda.',
          style: TextStyle(color: Colors.grey),
        ), // Text
      ) // Center
    : ListView.builder(
        itemCount: _alunos.length,
        itemBuilder: (context, index) {
          final aluno = _alunos[index];
          return Card(
            child: ListTile(
              title: Text(aluno.nome),
              subtitle: Text(
                'Idade: ${aluno.idade} | Curso: ${aluno.curso}', // Text
              ),
              trailing: IconButton(
                icon: const Icon(Icons.delete, color: Colors.red),
                onPressed: () => _removerAluno(index),
              ),
            ),
          );
        },
      ),
),
```

- Cada aluno é mostrado num Card com *ListTile*, e coloquei um botão de lixeira (bônus) para poder remover um aluno — não foi pedido, mas ajuda a testar o app.

```
final aluno = _alunos[index];
return Card(
  child: ListTile(
    title: Text(aluno.nome),
    subtitle: Text(
```